

## 正则表达式

Regular Expression, 使用单个字符串来描述、匹配句法规则的字符串

语法:

(1) 构造函数:

`let reg = new RegExp(模式串, 修饰符)`

(2) `let reg = /模式串/ 修饰符`

*i* 大小写不敏感

*g* 全局匹配而不是找到第一个就停止

*m* 多行匹配

通配符可选

### 常用的正则表达式

`[a-z]` 查找任何从小写a到小写z的字符

`[A-Z]` 查找任何从大写A到大写Z的字符

`[0-9]` 查找任何从0至9的数字

`[abc]` 查找括号内的任意一个字符

`[^abc]` 查找除括号内的任意字符

↪ 与 `^` 区分

表示以 `a/b/c` 中任一字符开头

### 常用的元字符(特殊字符)

`\w` 匹配数字、字母、下划线

`\W` 匹配非数字、非字母、非下划线

`\d` 匹配数字

`\D` 匹配非数字

`\s` 匹配空白字符(空格、换行)

`\S` 匹配非空白字符

`\n` 匹配换行符

### 常用的限定符

`*` 匹配前面的子表达式0次或多次

`+` 匹配前面的子表达式1次或多次

`?` 匹配前面的子表达式0次或1次

`{n}` 匹配确定的n次

`{n,}` 至少匹配n次

`{n,m}` 至少匹配n次且至多匹配m次

## 检测一个字符串是否与正则相匹配

例如.

```
const str = 'abc';  
let reg = /[a-z]/;  
console.log(reg.test(str)); ... true  
reg = /^(a-z)$/;  
console.log(reg.test(str)); ... false
```

没有^或\$ 就是“包含”匹配

有则要求完全匹配

验证26个英文字母组成的字符串 `/^[a-zA-Z]+$/`

验证数字和26个英文字母组成的字符串 `/^[a-zA-Z0-9]+$/`

验证由字母、数字、下划线组成的字符串 `/^\w+$/`

验证汉字 `/^[\\u4e00-\\u9fa5]+$/`

验证手机号码 `/^1[34578]\\d{9}$/`

验证邮箱 `/^\\w+@\\w+\\.com$|^cn)$/`

例. 登录功能表单验证

用户名: 6~20个字母、数字、下划线组成的字符串, 第一个是字母, 三种字符都要有

密码: 6~15个字母数字组合

插个话题, 几个验证的例子

姓名: 汉字, 长度2~10个 `/^[\\u4e00-\\u9fa5]{2,10}$/`

密码: 字母开头, 长度6~15, 仅包含数字、字母、下划线 `/^[a-zA-Z][a-zA-Z0-9_]{5,14}$/`

固话: 4位数字-7位任意数字 或 3位数字-8位任意数字

`/^(0\\d{3}-\\d{13}|0\\d{23}-\\d{83})$/`

## 窗口对象(WINDOW)

- ① BOM (浏览器对象模型)
- ② 定时器 - 延时函数
- ③ JS 执行机制
- ④ location 对象
- ⑤ navigation 对象
- ⑥ history 对象

window 对象是一个全局对象, 亦可说是 JavaScript 中的顶级对象

document, alert(), console.log 都是 window 的属性, 基本 BOM 的属性和方法都是 window 的

所有通过 var 定义在全局作用域中的变量、函数都会变成 window 对象的属性、方法  
window 对象下的属性和方法调用时可省略 window.

## location 对象

location 对象的数据类型是对象，它拆分并保存了 url 地址的各个组成部分

**href 属性** 获取完整的 url 地址，对其赋值时用于地址的跳转

```
console.log(location.href); // 获取当前文件的URL地址
```

```
location.href = 'https://www.ahut.edu.cn'; // 通过js跳转到目标地址
```

```
location.reload(); // 刷新, 相当于F5
```

```
location.reload(true); // 强制刷新, 相当于Ctrl + F5
```

例. 5秒后跳转到指定页面

```
setTimeout(function(){  
    location.href = 'https://www.ahut.edu.cn';  
}, 5000);
```

**search 属性** 获取地址中携带的参数，url 中 ? 后面的内容（即 Get 请求的参数部分）

**hash 属性** 获取地址中的哈希值，url 中 # 后面的内容。

## Navigator 对象

记录浏览器自身信息

## history 对象

管理历史对象，该对象与浏览器地址栏的操作相对应。

例如 前进、后退、历史记录等

```
back() // 后退
```

```
forward() // 前进
```

```
go(参数) // 为1则前进1个页面, 为-1则后退1个页面
```



## 本地存储

将数据存储在浏览器中, 设置读取方便, 甚至页面刷新不丢失数据, 容量较大

### localStorage

作用: 可将数据永远存储在本地, 除非手动删除, 不然关闭页面才存在

特性: 可以多窗口(页面)共享, 以键值对的形式存储使用。

只能存储字符串类型的数据, 相应的获取的数据也是字符串类型

`localStorage.setItem(key, value);` // 存储/修改数据 (有则改, 无则增)

`localStorage.getItem(key);` // 获取数据

`localStorage.removeItem(key);` // 删除数据

### sessionStorage

生命周期为关闭窗口

同一窗口(页面)下数据可以共享, 以键值对的形式存储使用

### 本地存储复杂数据类型(对象)

为解决本地只能存储字符串而不能存储复杂数据类型的问题:

{ 需要将复杂数据类型转换成JSON字符串, 再存储到本地

`JSON.stringify(复杂数据类型)`

JSON对象: 属性名和值有引号(统一是双引号)

{ 使用时, 将JSON字符串转换为对象

`JSON.parse(localStorage.getItem(key))`

例. `let obj = {  
 uname: 'zhangsan',  
 gender: '女',  
 age: 12  
};`

`localStorage.setItem('obj', JSON.stringify(obj));  
console.log(JSON.parse(localStorage.getItem('obj')));`

例. "记住我" (接上面表单验证的例子)

<script>

```
const uname=document.querySelector('input[type=text]');
const password=document.querySelector('input[type=password]');
const tip1=document.querySelector('.tip1');
const tip2=document.querySelector('.tip2');
const remember=document.querySelector('remember');
```

```
.tip1,.tip2 { margin-left: 10px;
               color: red;
}
.correct {
  color: green;
}
```

```
const reg1=/^[a-zA-Z]\w{5,19}$/;
const reg2=/[a-zA-Z]/;
const reg3=/\d/;
const reg4=/-/;
```

验证码

```
const reg5=/^[a-zA-Z0-9]{6,15}$/;
const reg6=/[a-zA-Z]/;
const reg7=/\d/;
```

```
function checkUname(){
  tip1.style.display='inline-block';
  const val=uname.value.trim();
  if(val.length===0){
    tip1.innerHTML='请输入用户名';
    tip1.className='tip1';
    return false;
  }
  if(reg1.test(val)&&reg2.test(val)
    &&reg3.test(val)&&reg4.test(val)){
    tip1.innerHTML='用户名正确';
    tip1.className='tip1 correct';
    return true;
  } else{
    tip1.innerHTML='用户名错误';
    tip1.className='tip1';
    return false;
  }
}
```

给密码加监听事件

```
function checkPwd(){
  tip2.style.display='inline-block';
  const val=password.value.trim();
  if(val.length===0){
    tip2.innerHTML='请输入密码';
    tip2.className='tip2';
    return false;
  }
  if(reg5.test(val)&&reg6.test(val)
    &&reg7.test(val)){
    tip2.innerHTML='密码正确';
    tip2.className='tip2 correct';
    return true;
  } else{
    tip2.innerHTML='密码错误';
    tip2.className='tip2';
    return false;
  }
}
```

password.addEventListener('blur', checkPwd);

function checkForm(){

```
const isUnameValid=checkUname;
const isPwdValid=checkPwd;
if(isUnameValid&&isPwdValid&&remember.checked){
  const userInfo={
    username:uname.value.trim(),
    password:password.value.trim(),
    saveTime:new Date().getTime();
  };
  localStorage.setItem('userInfo',JSON.stringify(userInfo));
} else if(!remember.checked){
  localStorage.removeItem('userInfo');
}
```

表单

验证

```
if(isUnameValid&&isPwdValid){
  alert('表单验证通过');
  return true;
} else{
  alert('表单中存在错误');
  return false;
}
```

```
loginForm.addEventListener('submit',function(){
  const isValid=checkForm();
  if(!isValid){
    e.preventDefault();
  }
});
```

这个方法是事件对象特有的

删除数据

将复杂数据类型转换成JSON字符串

存储数据

```
function loadSavedInfo () {  
    const savedInfo = localStorage.getItem('userInfo'); // 获取数据  
    if (savedInfo) { // 将JSON字符串转换为对象  
        const userInfo = JSON.parse(savedInfo);  
        // 显示在页面上的  
        username.value = userInfo.username;  
        password.value = userInfo.password;  
        remember.checked = true;  
    }  
}  
window.addEventListener('load', loadSavedInfo);
```

```
document.querySelector('input[type=reset]').addEventListener('click', function () {  
    tip1.style.display = 'none';  
    tip2.style.display = 'none';  
    tip1.className = 'tip1';  
    tip2.className = 'tip2';  
});  
</script>
```